

CONTENTS

Circuitos Secuenciales: Flip-Flops, Contadores, RAM y Máquinas de Estado	
Guía detallada desde cero — con los detalles que salvan el parcial	
0. Por qué existe este tema	
1. El Flip-Flop R-S (Latch) — de dónde sale todo	
2. Flip-Flop tipo D	
3. Flip-Flop tipo T	
4. Flip-Flop tipo J-K — el “universal”	
Tabla resumen — las tres ecuaciones características	
5. Construir un tipo de Flip-Flop a partir de otro	
5.1 Método — igualar ecuaciones características	
5.2 Tablas de excitación — el atajo para diseñar contadores y FSM	
6. Contadores	
6.1 Contador binario ascendente con FF T (versión “asíncrona”/ripple)	
6.2 Método síncronico general (con FF D) — contador binario módulo 4, resuelto y verificado	
6.3 Contador up/down (3 bits, con entrada UD), resuelto con Karnaugh completo	
6.4 Contador con carga paralela síncronica	
6.5 Contador de código Gray (3 bits, ciclo completo 0 a 7), con FF D	
6.6 Tabla resumen — tipos de contadores	
7. Memorias: RAM vs ROM, Latch vs Registro, SRAM vs DRAM	

7.1 RAM vs ROM	
7.2 Latch/Registro vs RAM propiamente dicha	
7.3 SRAM vs DRAM	
7.4 Tabla resumen — memorias	
8. Máquinas de Estado Finito (Autómatas): Moore vs Mealy	
8.1 Por qué formalizar con estados	
8.2 Moore vs Mealy — la diferencia clave	
8.3 Metodología completa de diseño	
8.4 Ejemplo completo — detector de secuencia “1101” (con solapamiento), Mealy, FF D	
8.5 Tabla resumen — Moore vs Mealy y la metodología	
Ejercicios de repaso rápido (con respuesta)	
Resumen ultra-rápido (2 minutos)	

Circuitos Secuenciales: Flip-Flops, Contadores, RAM y Máquinas de Estado

GUÍA DETALLADA DESDE CERO — CON LOS DETALLES QUE SALVAN EL PARCIAL

Arquitectura de Computadoras — FING UdelaR — Módulo 4

Flip-Flops D/T/JK · Tablas de excitación · Contadores (binarios, up/down, con carga paralela, Gray) · RAM vs ROM · SRAM vs DRAM · Máquinas de Estado (Moore/Mealy) · Diseño completo paso a paso

Cómo usar esta guía: cada tema tiene una definición corta, el procedimiento paso a paso, al menos dos ejemplos resueltos y verificados, una tabla resumen, y los errores de tolerancia cero que suelen costar puntos en el parcial. **Toda tabla de estados, tabla de excitación y ecuación de este documento fue generada y verificada por fuerza bruta con un script de Python (usando `sympy` para las simplificaciones booleanas y simulación bit a bit para las máquinas de estado)** — no hay ninguna tabla “de memoria” sin chequear contra su propia definición.

0. Por qué existe este tema

Hasta ahora (combinacional) toda salida se calculaba **solo** en función de las entradas actuales: un sumador, un multiplexor, un decodificador — apagás el circuito, lo volvés a prender con las mismas entradas, y da la misma salida, siempre. No hay “historia”.

Pero hay problemas que **no se pueden resolver así**. Un contador tiene que saber cuánto contó hasta ahora. Una memoria tiene que devolver lo último que se escribió, no una función de lo que se está pidiendo en este instante. Un detector de secuencia tiene que saber qué bits vinieron antes para decidir si el bit actual completa el patrón buscado.

Definición central: un circuito es **secuencial** si su salida depende no solo de la entrada actual, sino también de los valores anteriores de las entradas — equivalentemente, si tiene **memoria** (uno o más bits de **estado** que persisten entre un instante y el siguiente).

Formalmente (y esto se repite en todo el módulo, tomá nota):

$$Y = G(X, E) \quad E' = H(X, E)$$

donde X es la entrada actual, E el estado actual, E' el próximo estado, Y la salida. La diferencia entre un circuito combinacional y uno secuencial es exactamente la existencia de E : combinacional es el caso particular $Y=G(X)$, sin estado.

Pregunta que resuelve el 90% de los ítems teóricos de “combinacional vs secuencial”: ¿la salida de este circuito puede cambiar sin que cambie la entrada actual (por ejemplo, por el simple paso de un flanco de reloj)? Si sí → es secuencial (tiene estado). Si la salida solo puede cambiar cuando cambia la entrada → es combinacional.

Todo este módulo es, en el fondo, una sola idea aplicada en capas: el **Flip-Flop** es la celda mínima de estado (1 bit); el **contador** es varios Flip-Flops con lógica que los hace avanzar en una secuencia fija; la **RAM** es un arreglo de Flip-Flops (o su equivalente en capacitores) direccionable; la **Máquina de Estado** es el método general para diseñar cualquier circuito secuencial a medida, incluido el contador como caso particular.

1. El Flip-Flop R-S (Latch) — de dónde sale todo

Definición: el circuito secuencial más simple posible. Dos compuertas NOR realimentadas, con entradas R (Reset) y S (Set) y salidas Q , Q' .

Comportamiento:

R	S	$Q(t+1)$
0	0	$Q(t)$ — mantiene (memoria)
0	1	1
1	0	0
1	1	prohibido ($Q=Q'=0$, inconsistente al volver a 00)

La combinación $R=S=1$ está prohibida: fuerza $Q=Q'=0$ (viola que deben ser complementarias), y al volver ambas a 0 simultáneamente el resultado queda indeterminado (depende de cuál señal “gana la carrera” por microsegundos). Nunca se debe permitir que $R=S=1$ en un Latch R-S.

Por qué es secuencial (y no combinacional): con $R=S=0$, la salida no está determinada por la entrada actual — depende de si la última entrada distinta de (0,0) fue $R=1$ o $S=1$. Es decir, **el mismo par de entradas (0,0) produce dos salidas posibles según la historia**: esa es la

firma de un circuito secuencial.

R-S sincrónico: se le agrega una entrada de control G (habilitación/reloj) que hace que R y S solo sean “vistos” por el circuito mientras $G=1$. Si $G=0$, el circuito ignora R y S y retiene el valor. Esto es control **por nivel** (por oposición a control **por flanco**, que veremos abajo).

Nivel vs. flanco — la pregunta teórica más repetida: un FF sensible a nivel (tipo Latch con G) responde **mientras** la señal de control esté en el nivel activo (por ejemplo, todo el tiempo que $G=1$). Un FF sensible a flanco solo responde en el instante exacto de la transición de la señal de reloj ($0 \rightarrow 1$ flanco ascendente, o $1 \rightarrow 0$ descendente), ignorando cualquier cambio de las demás entradas durante el resto del período de reloj.

Set/Preset y Clear asincrónicos: en los FF reales por flanco suele haber dos entradas adicionales que actúan **por nivel** y de forma **asincrónica** (no esperan flanco de reloj): Preset (fuerza $Q=1$) y Clear (fuerza $Q=0$). Son asincrónicas porque su efecto es inmediato, sin importar el estado del reloj.

2. Flip-Flop tipo D

Definición: el más simple de usar. Tiene una sola entrada de datos D y una de reloj. En cada flanco activo, copia el valor de D a la salida.

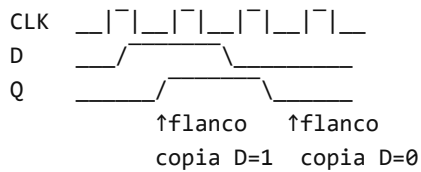
Ecuación característica:

$$Q_{n+1} = D_n$$

Tabla de verdad:

D	Q(t+1)
0	0
1	1

Diagrama de tiempo (control por flanco ascendente):



Q solo cambia en el instante del flanco ascendente de CLK; los cambios de D en cualquier otro momento no tienen efecto sobre Q.

Ejemplo 1 — calcular el próximo estado dado el estado actual y la entrada:

Un FF D tiene $Q(t)=0$. En el próximo flanco de reloj, $D=1$. ¿Cuál es $Q(t+1)$?

Aplico la ecuación característica: $Q(t+1) = D(t) = 1$. (El valor anterior de Q, 0, no interviene para nada — esa es la característica distintiva del FF D: no “recuerda” su propio valor, solo copia D.)

Ejemplo 2 — secuencia completa: $Q(0)=1$. Entradas D en los flancos sucesivos:

0,0,1,1,0.

$Q(1)=D=0$; $Q(2)=D=0$; $Q(3)=D=1$; $Q(4)=D=1$; $Q(5)=D=0$.

Secuencia de Q: 1,0,0,1,1,0.

Verificación rápida: en un FF D, $Q(t+1)$ es literalmente igual al valor de D en ese instante — nunca hace falta ninguna tabla, la ecuación es la identidad. Si en un ejercicio te da un resultado distinto de D, revisá el enunciado (probablemente el FF no es tipo D, o hay lógica combinacional extra antes de la entrada D).

Error frecuente: confundir “D” (la entrada del Flip-Flop) con “d” minúscula usada en las ecuaciones de excitación de un circuito secuencial completo (donde d_i es la señal que llega a la entrada D del FF i -ésimo, calculada por lógica combinacional a partir del estado y las entradas). Son cosas relacionadas pero no literalmente lo mismo: uno es el pin del FF, el otro es la ecuación booleana que lo alimenta.

3. Flip-Flop tipo T

Definición: tiene una entrada T (Toggle). Si $T=0$, mantiene su valor. Si $T=1$, invierte su valor en cada flanco activo.

Ecuación característica:

$$Q_{n+1} = Q_n T' + Q_n' T = Q_n \oplus T$$

Tabla de verdad:

Q _n	T	Q(t+1)
0	0	0
0	1	1
1	0	1
1	1	0

Ejemplo 1: Q(t)=0, T=1 en el flanco. Q(t+1) = Q ⊕ T = 0 ⊕ 1 = 1.

Ejemplo 2 — secuencia: Q(0)=0. Entradas T en flancos sucesivos: 1,1,0,1,1,1.

Q(1)=0 ⊕ 1=1; Q(2)=1 ⊕ 1=0; Q(3)=0 ⊕ 0=0; Q(4)=0 ⊕ 1=1; Q(5)=1 ⊕ 1=0; Q(6)=0 ⊕ 1=1.

Secuencia: 0,1,0,0,1,0,1. Notar que con T=1 constante el FF simplemente alterna 0,1,0,1,... a la mitad de la frecuencia del reloj — es exactamente el principio del contador binario asíncrono (Parte 6.1).

El FF T es, en esencia, un “divisor de frecuencia por 2” cuando T=1 siempre: por cada 2 flancos de reloj, Q completa un ciclo. Por eso la cascada de FF T (cada uno alimentado por la salida del anterior) construye directamente un contador binario.

4. Flip-Flop tipo J-K — el “universal”

Definición: generaliza y arregla el problema del R-S. Tiene dos entradas J\$ (como Set) y K\$ (como Reset), pero a diferencia del R-S, la combinación J=K=1 **no está prohibida**: produce el toggle (como el FF T).

Tabla de verdad:

J	K	Q(t+1)
0	0	Q(t) — mantiene
0	1	0
1	0	1
1	1	Q(t)' — invierte

Ecuación característica (verificada por fuerza bruta contra la tabla anterior — las 8 combinaciones de J,K,Q(t) reproducen exactamente $Q_{n+1} = JQ_n' + K'Q_n$):

$$Q_{n+1} = JQ_n' + K'Q_n$$

Por qué internamente funciona (resumen del análisis de la cascada de dos R-S

sincrónicos que forma el JK): el JK se arma con dos Latches R-S por nivel en cascada, con la habilitación del segundo negada respecto al primero. Esto asegura que las entradas del segundo FF nunca cambian mientras el primero está “escribiendo”, eliminando la carrera que hacía prohibido J=K=1 en el R-S simple; cuando J=K=1, el circuito queda armado para invertir la salida en cada flanco, sin ambigüedad.

Ejemplo 1 — calcular Q(t+1) dado el estado y las entradas:

Q(t)=1, J=0, K=1. Aplico la ecuación: $Q(t+1) = J \cdot Q' + K' \cdot Q = 0 \cdot 0 + 0 \cdot 1 = 0$. Coincide con la fila de la tabla (J=0,K=1 $\rightarrow$ Q(t+1)=0, sin importar Q(t)).

Ejemplo 2: Q(t)=0, J=1, K=1 (toggle). $Q(t+1) = 1 \cdot 1 + 0 \cdot 0 = 1$. Como J=K=1, invierte: pasó de 0 a 1 ✓.

Verificación cruzada rápida sin escribir la ecuación completa: si J=K, el JK se comporta como un FF T con T=J=K (mantiene si ambos 0, invierte si ambos 1). Si J=D y K=D' (complementarios), el JK se comporta como un FF D. Esto sirve para chequear resultados sin rehacer toda la tabla.

Error de tolerancia cero: la ecuación es $Q_{n+1} = JQ_n' + K'Q_n$, con Q_n' multiplicando a J y Q_n (sin negar) multiplicando a K' . Es un error muy común escribirla al revés ($JQ_n + K'Q_n'$) — verifícala siempre con la fila J=0,K=0 (debe dar Q(t+1)=Q(t), es decir, mantiene): con la ecuación correcta, J=0,K=0 $\rightarrow 0 \cdot Q' + 1 \cdot Q = Q$ ✓; con la versión invertida, J=0,K=0 $\rightarrow 0 \cdot Q + 1 \cdot Q' = Q'$ X (invertiría el valor, lo cual está mal).

TABLA RESUMEN — LAS TRES ECUACIONES CARACTERÍSTICAS

FF	Ecuación característica	Comportamiento
D	$Q_{n+1} = D_n$	copia la entrada
T	$Q_{n+1} = Q_n T' + Q_n' T = Q_n \oplus T$	mantiene (T=0) / invierte (T=1)
JK	$Q_{n+1} = JQ_n' + K'Q_n$	mantiene(00) / reset(01) / set(10) / invierte(11)

5. Construir un tipo de Flip-Flop a partir de otro

Idea general: dado un FF “base” disponible (por ejemplo, tipo D) y otro tipo que queremos “emular” (por ejemplo, JK), buscamos qué señal hay que poner en la entrada del FF base para que su ecuación característica coincida, entrada a entrada, con la ecuación característica del FF que queremos construir.

5.1 MÉTODO — IGUALAR ECUACIONES CARACTERÍSTICAS

Procedimiento:

1. Escribir la ecuación característica del FF deseado (el que queremos “construir”), en función de sus entradas y de Q_n .
2. Escribir la ecuación característica del FF disponible (el que tenemos físicamente).
3. Igualar ambas expresiones de Q_{n+1} y despejar la entrada del FF disponible.
4. El resultado es una expresión combinacional (compuertas) que hay que anteponer a la entrada del FF disponible, alimentada con las entradas del FF deseado.

Ejemplo resuelto — construir un FF JK a partir de un FF D (este es el ejercicio clásico del Práctico 5, ítem (c)):

Ecuación característica JK (deseado): $Q_{n+1} = JQ_n' + K'Q_n$

Ecuación característica D (disponible): $Q_{n+1} = D_n$

Igualando: $D_n = JQ_n' + K'Q_n$

Conclusión: basta con armar el circuito combinacional $D = J\overline{Q} + \overline{K}Q$ (dos AND + un OR + dos NOT, usando la propia salida Q realimentada) y conectarlo a la entrada D de un FF D físico. El resultado se comporta exactamente como un FF JK.

Verificación (hecha por fuerza bruta en Python, sympy): se generaron las 8 combinaciones de (J,K,Q) según la tabla JK, y se comprobó que evaluando $D = J \cdot Q' + K' \cdot Q$ y aplicando la

regla del FF D ($Q(t+1)=D$) se reproduce exactamente el mismo $Q(t+1)$ que la tabla JK original, para las 8 filas sin excepción. Chequeo manual de una fila: $J=1, K=1, Q=1$ (debería invertir $\rightarrow Q(t+1)=0$). $D = 1 \cdot 0 + 0 \cdot 1 = 0$. FF D copia $D \rightarrow Q(t+1)=0$ ✓.

5.2 TABLAS DE EXCITACIÓN — EL ATAJO PARA DISEÑAR CONTADORES Y FSM

Cuando diseñamos un contador o una máquina de estados con un tipo de FF dado, no partimos de “tengo J,K, quiero D”: partimos al revés — **conocemos la transición deseada $Q(t) \rightarrow Q(t+1)$ y necesitamos saber qué valor poner en las entradas del FF para lograrla.** Eso es la tabla de excitación (verificada por fuerza bruta invirtiendo cada ecuación característica):

FF D (no hay ambigüedad: D es literalmente el próximo valor deseado):

Q(t)	Q(t+1)	D
0	0	0
0	1	1
1	0	0
1	1	1

FF T ($T=1$ exactamente cuando $Q(t)$ y $Q(t+1)$ difieren):

Q(t)	Q(t+1)	T
0	0	0
0	1	1
1	0	1
1	1	0

FF JK (acá SÍ hay libertad — combinaciones “don’t care” X, porque tanto J como K tienen más de una forma de producir la misma transición):

Q(t)	Q(t+1)	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

Ejemplo de uso — diseñar con FF JK una transición $Q(t)=1 \rightarrow Q(t+1)=0$: según la tabla de excitación, fila $Q=1, Q(t+1)=0 \rightarrow J=X, K=1$. Esto significa: para forzar ese “apagado” no importa qué valor tenga J (podés dejarlo en 0 o en 1, el resultado es el mismo porque $K=1$ domina), pero K debe valer necesariamente 1. Estos “X” son los que después se aprovechan como comodines en el Karnaugh para simplificar más el circuito (Parte 6.3).

Error frecuente de tolerancia cero: tratar los “X” de la tabla de excitación JK como si fueran “0 conocido”. Un X es un comodín real — se debe usar como tal en el Karnaugh (agrupándolo con 1s o con 0s, lo que convenga para simplificar), nunca fijarlo arbitrariamente a 0 antes de hacer el mapa.

6. Contadores

Definición: un contador es un circuito secuencial cuyas salidas representan, en un código dado (binario, Gray, BCD, etc.), la cantidad de flancos activos recibidos, módulo su capacidad. No necesariamente cuenta en potencias de 2 ni empieza en 0.

6.1 CONTADOR BINARIO ASCENDENTE CON FF T (VERSIÓN “ASÍNCRONA”/RIPPLE)

Idea: cada FF T se alimenta con $T=1$ fijo (siempre invierte), y el reloj de cada FF (salvo el primero) es la salida Q del FF anterior.

- FF0: cambia en cada flanco de la entrada (toggle constante).
- FF1: cambia cada vez que Q0 pasa de 1 a 0 (o de 0 a 1, según el sentido elegido).
- FF_i: cambia cuando el FF anterior completa un ciclo.

Con 4 FF T conectados así ($T=1$ fijo, reloj de cada uno = Q del anterior), tras 16 flancos el circuito vuelve a la condición inicial: es un **contador binario módulo 16**. Si se invierte el sentido de conexión (usar Q' en vez de Q como reloj del siguiente), se obtiene la cuenta ascendente en vez de descendente.

Este contador se llama “asíncrono” o “ripple” porque los FF no cambian todos en el mismo instante: el cambio se propaga en cascada de FF en FF, con un pequeño retardo acumulado. Por eso, para diseños sincrónicos “limpios” (todos los FF cambian exactamente en el mismo flanco del reloj global), se prefiere el método de Máquina de Estado con Karnaugh que vemos a continuación.

6.2 MÉTODO SINCRÓNICO GENERAL (CON FF D) — CONTADOR BINARIO MÓDULO 4, RESUELTO Y VERIFICADO

Este es el método que se generaliza a cualquier contador (con o sin entradas, con carga, con o sin código binario puro).

Procedimiento:

1. Tabla de estados: para cada estado actual, cuál es el próximo estado (y la salida, si aplica).
2. Asignar codificación binaria a los estados (cantidad de FF = $\lceil \log_2(\text{cantidad de estados}) \rceil$).
3. Tabla de transiciones (estado actual codificado $\rightarrow$ estado siguiente codificado).
4. Usando la ecuación característica del FF elegido, pasar de “próximo estado deseado” a “entrada que hay que poner en cada FF”.
5. Karnaugh para cada entrada de FF (y para cada salida, si no coincide directamente con el estado).
6. Circuito final.

Ejemplo completo — contador binario módulo 4 (0,1,2,3,0,...), sin entrada, con FF D:
2 bits de estado ($q_1 q_0$), la salida se define igual al estado.

Tabla de transiciones (verificada: cada fila = estado actual $\rightarrow$ estado siguiente = estado+1 mod 4):

q1 q0 (estado)	próximo estado	D1	D0
00	01	0	1
01	10	1	0
10	11	1	1
11	00	0	0

Como el FF es tipo D, D_i = próximo valor de q_i directamente (ecuación característica $Q_{n+1} = D_n$), así que la tabla de arriba ya son los valores a llevar al Karnaugh.

Karnaugh de D1 (variables q_1, q_0): 1 en las filas 01 y 10 $\rightarrow$ agrupando: $D_1 = q_1 q_0' + q_1' q_0$ (XOR de q_1, q_0).

Karnaugh de D0: 1 en las filas 00 y 10 ($q_0=0$ en ambas) $\rightarrow D_0 = q_0'$.

Verificación (Python/sympy, minimización SOP automática + chequeo fila por fila contra la tabla de transiciones):

$D_1 = (q_0 \wedge \neg q_1) \vee (q_1 \wedge \neg q_0)$ — coincide con $XOR(q_1, q_0)$ ✓

$D_0 = \neg q_0$ ✓

Las 4 filas de la tabla se reprodujeron exactamente al evaluar estas dos ecuaciones.

Chequeo manual rápido sin rehacer el Karnaugh: en un contador binario puro, el bit menos significativo siempre se invierte en cada pulso ($D_0 = q_0'$), y cada bit i -ésimo se invierte exactamente cuando todos los bits menos significativos están en 1 (para contador con FF T sería $T_i = q_{i-1} \cdots q_0$). Si tu D_0 no da $\overline{q_0}$, algo está mal.

6.3 CONTADOR UP/DOWN (3 BITS, CON ENTRADA UD), RESUELTO CON KARNAUGH COMPLETO

Especificación: entrada UD (1=incrementa, 0=decrementa). Con FF D. 3 bits de estado $q_2 q_1 q_0$.

Tabla de transiciones (16 filas: 8 estados $\times$ 2 valores de UD — generada exhaustivamente y verificada):

UD	q2 q1 q0	próximo estado	D2 D1 D0
1	000	001	0 0 1
1	001	010	0 1 0
1	010	011	0 1 1
1	011	100	1 0 0
1	100	101	1 0 1
1	101	110	1 1 0
1	110	111	1 1 1
1	111	000	0 0 0
0	000	111	1 1 1
0	001	000	0 0 0
0	010	001	0 0 1
0	011	010	0 1 0
0	100	011	0 1 1
0	101	100	1 0 0
0	110	101	1 0 1
0	111	110	1 1 0

Ecuaciones minimizadas (Karnaugh de 4 variables UD,q2,q1,q0 para cada D_i — obtenidas y verificadas con sympy contra las 16 filas de arriba):

$$\begin{aligned}
 D_0 &= q_0' \\
 D_1 &= q_1 q_0 (UD)' + q_0 (UD) q_1' + q_1 (UD) q_0' + q_1' q_0' (UD)' \\
 D_2 &= q_0 q_2 (UD)' + q_1 q_2 q_0' + q_2 (UD) q_1' + q_0 q_1 (UD) q_2' + q_0' q_1' q_2' (UD)'
 \end{aligned}$$

Estas ecuaciones (obtenidas por Karnaugh directo) se verifican también con la forma equivalente basada en FF T, mucho más fácil de razonar e igual de válida (se demostró por simulación que produce exactamente los mismos D_i tras aplicar $D_i = Q_i \oplus T_i$): $T_0 = 1$, $T_1 = UD \odot q_0$ (XNOR), $T_2 = UD \odot q_1 q_0 + UD' \odot q_1' q_0'$. Es decir: el bit 0 siempre “cuenta”; el bit i-ésimo (i>0) invierte

su valor cuando “hay acarreo/préstamo desde los bits menos significativos” — todos ellos en 1 si se está subiendo, o todos en 0 si se está bajando. Esta es la forma que conviene memorizar para razonar rápido en el parcial; el Karnaugh de arriba es la forma “de manual” que hay que saber producir si piden el desarrollo completo.

Error muy común: al construir el Karnaugh de un contador up/down, olvidarse de que la variable UD también entra al mapa (mapa de 4 variables, no de 3) — sin UD como variable de entrada, el Karnaugh no puede distinguir si $Q_1Q_0=011$ debe ir a 100 (subiendo) o a 010 (bajando), y el resultado simplificado sale mal o inconsistente.

6.4 CONTADOR CON CARGA PARALELA SINCRÓNICA

Especificación: entrada LOAD y entradas de datos I_1, I_0 (2 bits). Si $LOAD=1$, en el próximo flanco el contador adopta el valor ($I_1 I_0$) sin importar el estado actual. Si $LOAD=0$, cuenta normalmente (módulo 4, ascendente). FF D.

Tabla de transiciones (resumen; generada y verificada exhaustivamente, 20 combinaciones): cuando $LOAD=1$, $D_1D_0 = I_1I_0$ directamente; cuando $LOAD=0$, $D_1D_0 = (q_1q_0)+1 \bmod 4$ (igual que la Parte 6.2).

Ecuaciones minimizadas (verificadas con sympy):

$$\begin{aligned} D_1 &= (LOAD \cdot I_1) + (LOAD' \cdot q_0 \cdot q_1') + (LOAD' \cdot q_1 \cdot q_0') \\ D_0 &= (LOAD \cdot I_0) + (LOAD' \cdot q_0') \end{aligned}$$

Forma de leer la ecuación sin memorizarla: cada D_i es un multiplexor 2 a 1 controlado por $LOAD$ — si $LOAD=1$ pasa el dato paralelo I_i , si $LOAD=0$ pasa la lógica normal de “contar”. Es literalmente $D_i = LOAD \cdot I_i + LOAD' \cdot (\text{lógica de contar})_i$. Cualquier contador con carga se puede razonar así sin rehacer el Karnaugh completo.

6.5 CONTADOR DE CÓDIGO GRAY (3 BITS, CICLO COMPLETO 0 A 7), CON FF D

Los contadores no tienen por qué usar código binario puro. El código Gray cambia **un solo bit** entre estados consecutivos (útil para evitar errores de lectura en sistemas mecánicos/eléctricos donde varios bits cambiando “casi simultáneo” pueden leerse mal).

Secuencia Gray de 3 bits: 000, 001, 011, 010, 110, 111, 101, 100, (vuelve a 000).

Tabla de transiciones completa (verificada exhaustivamente contra la secuencia anterior):

q2 q1 q0	próximo estado	D2 D1 D0
000	001	0 0 1
001	011	0 1 1
011	010	0 1 0
010	110	1 1 0
110	111	1 1 1
111	101	1 0 1
101	100	1 0 0
100	000	0 0 0

Ecuaciones minimizadas (Karnaugh, verificadas con sympy contra las 8 filas):

$$D_2 = q_0 q_2 + q_1 q_0' \quad D_1 = q_0 q_2' + q_1 q_0' \quad D_0 = q_1 q_2 + q_1' q_2'$$

Chequeo manual de una fila: estado 011 ($q_2=0, q_1=1, q_0=1$). $D_2 = q_0 \cdot q_2 + q_1 \cdot q_0' = 1 \cdot 0 + 1 \cdot 0 = 0$. $D_1 = q_0 \cdot q_2' + q_1 \cdot q_0' = 1 \cdot 1 + 1 \cdot 0 = 1$. $D_0 = q_1 \cdot q_2 + q_1' \cdot q_2' = 1 \cdot 0 + 0 \cdot 1 = 0$.

Resultado: $D_2 D_1 D_0 = 010$, es decir el próximo estado es 010 ✓ (coincide con la tabla: 011 → 010).

6.6 TABLA RESUMEN — TIPOS DE CONTADORES

Tipo	Entrada de control	FF típico	Idea clave
Binario ascendente/descendente (ripple)	ninguna	T	cascada, reloj de cada FF = Q (o Q') del anterior
Binario síncronico	ninguna	D o T	Karnaugh estándar, $D_i = q_{i+1} \pmod N$
Up/Down	UD	D o T	Karnaugh con UD como variable extra, o forma T con XNOR/carry
Con carga paralela	LOAD + datos	D	$D_i = \text{LOAD} \cdot \text{dato}_i + \text{LOAD}' \cdot (\text{lógica de contar})$
Código Gray / no binario	ninguna o la que corresponda	D	tabla de transiciones ad-hoc según la secuencia deseada, luego Karnaugh igual que siempre

Error de examen frecuente: asumir que “más estados = más Flip-Flops de forma automática proporcional”. La cantidad de FF es $\lceil \log_2(\text{cantidad de estados}) \rceil$, redondeando hacia arriba. Un contador con 10 estados (0 a 9) necesita 4 FF, no 3.33 — y sobran 6 combinaciones de código no usadas, que se tratan como “don't care” en el Karnaugh (Parte 8 profundiza esto para máquinas de estado en general).

7. Memorias: RAM vs ROM, Latch vs Registro, SRAM vs DRAM

7.1 RAM VS ROM

ROM (Read-Only Memory): memoria de solo lectura — el contenido se fija en fabricación o programación previa y no cambia durante la operación normal. Es un circuito **combinacional**: dado un valor de dirección, siempre devuelve el mismo dato (no tiene “escritura” que altere ese comportamiento en tiempo de uso).

RAM (Random Access Memory): memoria de lectura **y escritura**. Es un circuito **secuencial**: si escribo un valor en una posición y después la leo, obtengo el valor que escribí — el contenido de una dirección puede cambiar con el tiempo según lo que se haya escrito, es decir, tiene **memoria** de lo que pasó antes (exactamente la definición de la Parte 0).

Respuesta modelo a la pregunta teórica clásica “¿una RAM es combinacional o secuencial? ¿por qué?”: es secuencial, porque su salida (el dato leído) no depende solo de la entrada actual (la dirección), sino también de la historia de escrituras anteriores sobre esa misma dirección — y porque internamente está construida con Flip-Flops (o el equivalente en capacitores para DRAM), que son por definición elementos de memoria.

7.2 LATCH/REGISTRO VS RAM PROPIAMENTE DICHA

	Latch / Registro	RAM
Cantidad de palabras	una sola palabra de n bits	2^m palabras de n bits
Necesita dirección	no	sí (m bits de dirección)
Circuito interno	conjunto de FF D sincrónicos con reloj común	FF D (o capacitor) por bit, con decodificador de dirección que habilita la palabra seleccionada

7.3 SRAM VS DRAM

SRAM (Static RAM): cada bit se guarda en un **Flip-Flop** (biestable realimentado). Mientras tenga alimentación, retiene el dato indefinidamente sin necesidad de ninguna acción periódica — de ahí “estática”. Es más rápida, pero requiere más transistores por bit (típicamente 6), por lo que es más cara y menos densa.

DRAM (Dynamic RAM): cada bit se guarda como **carga eléctrica en un capacitor** (parásito de un único transistor de acceso). Requiere solo 1 transistor por bit → mucha mayor densidad y menor costo por bit, pero:

Por qué la DRAM tiene “lectura destructiva”: leer un bit de una DRAM significa literalmente detectar (y drenar) la carga acumulada en el capacitor para generar la corriente que el sensor de lectura interpreta como 0 o 1. Ese mismo acto de leer descarga el capacitor. Por lo tanto, inmediatamente después de leer, el circuito interno debe **volver a escribir** (regrabar) el mismo valor que se acaba de leer, o la información se pierde. Esta regrabación automática la hace el propio chip, de forma transparente para quien diseña el sistema que usa la memoria.

Por qué necesita “refresco” periódico incluso sin leerla: ningún capacitor es un aislante perfecto — existen corrientes de fuga que hacen que la carga almacenada se disipe

con el tiempo, incluso sin que nadie la lea. Si no se hace nada, al cabo de unos milisegundos la carga (y con ella el bit) desaparece. Por eso hay que leer (y así regrabar) cada celda periódicamente, con un período menor al tiempo de fuga, aunque nadie esté usando esos datos en ese momento — a esto se le llama “refresh”.

Truco para no confundir SRAM/DRAM en un ítem de examen: SRAM = “S” de FF (bi-estable, sin fuga mientras haya alimentación, sin refresco). DRAM = “D” de capacitor con fuga (necesita “D”espertarse/regrabarse periódicamente = refresh). La lectura destructiva y el refresco son exclusivos de DRAM — nunca de SRAM.

Organización matricial de la DRAM: para minimizar la cantidad de pines y el tiempo de refresco, las direcciones se dividen en fila (RAS, Row Address Strobe) y columna (CAS, Column Address Strobe), reutilizando los mismos pines de dirección en dos tiempos distintos. Esto permite refrescar toda una fila (todas sus columnas) en una sola operación de lectura de fila, reduciendo drásticamente el número de accesos necesarios para refrescar el chip completo (de 2^m a $2^{m/2}$ accesos, aproximadamente, para m bits de dirección).

7.4 TABLA RESUMEN — MEMORIAS

	ROM	Latch/Registro	SRAM	DRAM
Lectura/Escritura	solo lectura	ambas	ambas	ambas
Combinacional/Secuencial	combinacional	secuencial	secuencial	secuencial
Elemento de memoria	— (fijo)	FF D	FF D	capacitor + transistor
Múltiples palabras	sí	no (1 palabra)	sí	sí
Necesita refresco	no	no	no	sí
Lectura destructiva	no	no	no	sí
Densidad / costo	—	—	menor densidad, más cara	mayor densidad, más barata

Error frecuente: decir que la ROM “no tiene memoria”. Sí tiene — recuerda su contenido — pero ese contenido no cambia durante la operación (no se escribe en tiempo de uso nor-

mal), por lo que su comportamiento de lectura es indistinguible de un circuito combinacional puro visto desde afuera: misma dirección, siempre mismo dato.

8. Máquinas de Estado Finito (Autómatas): Moore vs Mealy

8.1 POR QUÉ FORMALIZAR CON ESTADOS

Un circuito secuencial puede tener, en teoría, infinitas secuencias de entrada posibles. Para poder diseñarlo con un método sistemático, agrupamos esas secuencias en clases de equivalencia — dos secuencias son “equivalentes” si a partir de cierto punto producen la misma salida en adelante. Cada clase de equivalencia es un **estado**. Si el número de estados distintos es finito, el circuito se puede modelar (y diseñar) como un **Autómata Finito Determinista (AFD)**.

8.2 MOORE VS MEALY — LA DIFERENCIA CLAVE

Máquina de Mealy: la salida depende del **estado actual Y de la entrada actual**.

$$Y = G(X, E) \quad E' = H(X, E)$$

Máquina de Moore: la salida depende **solo del estado actual** (no de la entrada actual).

$$Y = G(E) \quad E' = H(X, E)$$

Regla mnemotécnica de tolerancia cero: en **Moore** la salida se pinta dentro del círculo del **estado** en el diagrama (una sola salida por estado, no depende de por dónde se llegó ni de la entrada actual). En **Mealy** la salida se escribe sobre la **flecha** de transición (junto con la entrada que la dispara: “entrada/salida”). Si en el diagrama ves “x/y” sobre los arcos → Mealy. Si ves la salida escrita adentro del círculo del estado → Moore.

Consecuencia práctica que se pregunta seguido: en una máquina de Mealy la salida puede cambiar en el instante en que cambia la entrada (sin esperar flanco de reloj, si la implementación es puramente combinacional para la salida), mientras que en Moore la salida solo puede cambiar cuando cambia el estado, es decir, sincronizada con el flanco de reloj que actualiza los FF. Esto hace a Moore más “predecible” en el tiempo pero típicamente requiere uno o más estados extra comparado con el diseño Mealy equivalente.

8.3 METODOLOGÍA COMPLETA DE DISEÑO

1. **Diagrama/tabla de estados** a partir del enunciado: identificar qué necesita “recordar” el circuito, definir un estado por cada situación distinguible, y las transiciones para **todas** las combinaciones de entrada desde **todos** los estados.
2. **Codificación binaria** de los estados: $\lceil \log_2(\#\text{estados}) \rceil$ Flip-Flops. Elegir una asignación (arbitraria salvo que convenga alguna por simplicidad de las ecuaciones resultantes).
3. **Tabla de transiciones y salidas**: reemplazar los nombres de estado por su código binario, listando para cada combinación (estado codificado, entrada) el (próximo estado codificado, salida).
4. **Ecuaciones de excitación**: usando la ecuación característica del FF elegido (o su tabla de excitación si es JK), pasar de “próximo estado deseado” a “qué hay que poner en cada entrada de FF”.
5. **Karnaugh** para cada entrada de FF y para la(s) salida(s) — aprovechando como “don’t care” los códigos binarios que no corresponden a ningún estado real (si el número de estados no es potencia de 2).
6. **Circuito final** con FF + compuertas AND/OR/NOT.

8.4 EJEMPLO COMPLETO — DETECTOR DE SECUENCIA “1101” (CON SOLAPAMIENTO), MEALY, FF D

Enunciado: diseñar un circuito con una entrada serial x que produzca $Y=1$ exactamente en el instante en que los últimos 4 bits recibidos formaron el patrón “1101” (permitiendo solapamiento: por ejemplo, en la entrada 1101101 debe detectar dos veces).

Paso 1 — diagrama/tabla de estados. Defino:

- S0: no hay ningún prefijo útil del patrón reconocido todavía.
- S1: el último bit relevante visto es “1” (posible comienzo).
- S2: se vio “11”.
- S3: se vio “110”.

Análisis exhaustivo de cada estado con cada entrada (0 y 1) — este es el paso que más se salta y el que más puntos cuesta si se hace mal:

Estado	$x=0 \rightarrow$ (próximo, Y)	$x=1 \rightarrow$ (próximo, Y)
S0	(S0, 0)	(S1, 0)
S1	(S0, 0)	(S2, 0)
S2	(S3, 0)	(S2, 0) — “111”: los últimos dos siguen siendo “11”
S3	(S0, 0)	(S1, 1) — completó “1101”; el “1” que llega es a la vez el fin del patrón y el posible comienzo de uno nuevo

Paso 2 — codificación. 4 estados $\rightarrow$ 2 FF ($q_1 q_0$). Asigno: S0=00, S1=01, S2=11, S3=10.

Paso 3 — tabla de transiciones y salidas (verificada exhaustivamente por simulación bit a bit contra la especificación anterior, incluyendo 2000 tiras de entrada aleatorias de hasta 12 bits — 100% de coincidencia):

Estado	$q_1 q_0$	x	próximo	D1 D0	Y
S0	00	0	S0	0 0	0
S0	00	1	S1	0 1	0
S1	01	0	S0	0 0	0
S1	01	1	S2	1 1	0
S2	11	0	S3	1 0	0
S2	11	1	S2	1 1	0
S3	10	0	S0	0 0	0
S3	10	1	S1	0 1	1

Paso 4 — ecuaciones (FF D, así que D_i = próximo valor de q_i directamente).

Paso 5 — Karnaugh (3 variables: q_1, q_0, x) y minimización, verificadas con sympy contra las 8 filas de la tabla:

$$D_1 = q_1 q_0 + q_0 x \quad D_0 = x \quad Y = q_1 \cdot x \cdot q_0$$

Verificación manual de una fila (para practicar el chequeo, no solo confiar en el script): estado S2 ($q_1=1, q_0=1$), $x=0$.

$D1 = q1q0 + q0x = 1 \cdot 1 + 1 \cdot 0 = 1$. $D0 = x = 0$. $Y = q1 \cdot x \cdot q0' = 1 \cdot 0 \cdot 0 = 0$.

Resultado: $D1D0=10 \rightarrow$ próximo estado = 10 = S3, $Y=0$. Coincide exactamente con la fila de la tabla ($S2, x=0 \rightarrow S3, Y=0$) ✓.

Verificación del caso “gancho” del solapamiento — estado S3 ($q1=0, q0=1$)... espera, ojo: $S3=10$, o sea $q1=1, q0=0$. Con $x=1$: $D1=q1q0+q0x=1 \cdot 0+0 \cdot 1=0$. $D0=x=1$.

$Y=q1 \cdot x \cdot q0'=1 \cdot 1 \cdot 1=1$.

Resultado: $D1D0=01 \rightarrow$ próximo estado= $01=S1$, $Y=1$. Coincide con la fila ($S3, x=1 \rightarrow S1, Y=1$) ✓ — el circuito detecta el patrón y a la vez reconoce que el “1” que acaba de llegar puede ser el comienzo de una nueva ocurrencia solapada.

Prueba de estrés que se hizo para esta guía (y que conviene hacer siempre que el tiempo del parcial lo permita, al menos mentalmente con 2-3 tiras cortas): simular el circuito derivado ($D1, D0, Y$ en función de $q1, q0, x$) bit a bit sobre una tira de entrada concreta y comparar con lo que uno esperaría “a mano” siguiendo el diagrama de estados. Por ejemplo, con entrada 1,1,0,1,1,0,1 el circuito enciende $Y=1$ en la posición 4 (fin de “1101”) y otra vez en la posición 7 (fin del “1101” solapado que empieza en la posición 4) — exactamente lo que predice el diagrama.

Error de tolerancia cero en el diseño de FSM: no analizar ****todas**** las combinaciones (estado, entrada). Es muy común, bajo presión de tiempo, dibujar solo las transiciones “que importan” para el caso feliz y dejar sin definir, por ejemplo, qué pasa en S2 si llega un 0 dos veces seguidas. Un AFD completo necesita, para cada estado, una transición definida para cada símbolo posible del alfabeto de entrada — sin excepciones.

8.5 TABLA RESUMEN — MOORE VS MEALY Y LA METODOLOGÍA

	Mealy	Moore
Salida depende de	estado y entrada actual	solo del estado actual
Notación en el diagrama	"entrada/salida" sobre el arco	salida dentro del círculo del estado
Cambios de salida	pueden ocurrir sin esperar flanco (si Y es puramente combinacional)	solo en flancos de reloj (cuando cambia el estado)
Cantidad típica de estados	suele necesitar menos estados	suele necesitar 1 estado extra por cada "versión con distinta salida" del mismo estado Mealy

La Máquina Lógica General: cualquier máquina de Mealy se puede implementar con una ROM (que calcula G y H combinadas) más una RAM tipo registro (que guarda el estado E) realimentada. Sustituyendo la ROM por otra RAM programable se obtiene la "Máquina Lógica General Programable" — la idea germinal de qué es, en el fondo, una computadora: una máquina de estados universal cuyo comportamiento específico se define "programando" el contenido de esa memoria.

Ejercicios de repaso rápido (con respuesta)

#	Enunciado	Respuesta
1	FF JK con $Q(t)=1$, $J=1$, $K=0$. ¿ $Q(t+1)$?	$Q(t+1)=J \cdot Q' + K' \cdot Q = 1 \cdot 0 + 1 \cdot 1 = 1$ (set, coherente con la fila $J=1, K=0 \rightarrow 1$)
2	FF T con $Q(t)=1$, $T=1$. ¿ $Q(t+1)$?	0 (toggle: invierte)
3	Tabla de excitación JK para la transición $0 \rightarrow 1$. ¿J y K?	J=1, K=X
4	¿Cuántos FF mínimos hacen falta para un contador de 10 estados (0 a 9)?	4 ($2^3=8$ no alcanza; $2^4=16$ sí, sobran 6 códigos como don't care)
5	¿La combinación de entradas $R=S=1$ en un Latch R-S está permitida?	No , es la combinación prohibida (indeterminación al volver a 0,0)
6	¿Qué tipo de memoria tiene lectura destructiva y necesita refresco?	DRAM
7	¿Cuántos transistores por bit necesita típicamente una SRAM vs una DRAM?	SRAM ≈ 6 , DRAM ≈ 1
8	En una máquina de Moore, ¿de qué depende la salida?	Solo del estado actual (no de la entrada actual)
9	En el diagrama de estados, ¿dónde se escribe la salida en una máquina de Mealy?	Sobre el arco de transición (junto a la entrada que la dispara)
10	Dado $D=J \cdot Q' + K' \cdot Q$ conectado a un FF D, ¿qué tipo de FF se emula?	JK
11	En un contador up/down con FF D, ¿qué variable extra hay que incluir siempre en el Karnaugh además de los bits de estado?	La entrada de control UD (up/down)
12	En un contador con carga paralela, si $LOAD=1$, ¿de qué depende D_i ?	Del dato paralelo I_i (ignora la lógica de contar)
13	¿Un FF sensible a flanco responde a cambios de sus entradas durante todo el período en que el reloj vale 1?	No , solo en el instante exacto del flanco (activo)
14	¿Una memoria ROM es combinacional o secuencial?	Combinacional (misma dirección $\rightarrow$ siempre mismo dato, no hay escritura en

#	Enunciado	Respuesta
		uso normal)
15	Tabla de excitación D para la transición $1 \rightarrow 0$. ¿D?	D=0

Resumen ultra-rápido (2 minutos)

- **Secuencial** = la salida depende de la entrada actual **y del estado** (memoria); **combinacional** = solo de la entrada actual. $Y=G(X,E)$, $E'=H(X,E)$.
- **D**: $Q_{n+1}=D$. **T**: $Q_{n+1}=Q \oplus T$ (mantiene/invierte). **JK**: $Q_{n+1}=JQ' + K'Q$ (mantiene/reset/set/invierte); $J=K=1$ no está prohibido (a diferencia de $R=S=1$ en el Latch R-S).
- Para construir un FF a partir de otro: **igualar** las dos ecuaciones características y despejar la entrada del FF disponible. ($D=JQ' + K'Q$ da un JK usando FF D).
- Tablas de excitación: D es directo ($D=Q(t+1)$); T=1 si $Q(t) \neq Q(t+1)$; JK tiene comodines X (más de una forma de lograr la misma transición).
- **Contador**: tabla de transiciones $\rightarrow$ ecuación característica del FF $\rightarrow$ Karnaugh de cada D_i (o $T_i/J_i/K_i$) $\rightarrow$ circuito. Up/down agrega la variable de control al mapa. Carga paralela: $D_i = \text{LOAD} \cdot I_i + \text{LOAD}' \cdot (\text{contar})_i$.
- **ROM**=solo lectura=combinacional. **RAM**=lectura y escritura=secuencial. **SRAM**=Flip-Flops, rápida, sin refresco. **DRAM**=capacitor+1 transistor, barata y densa, **lectura destructiva** (leer descarga $\rightarrow$ hay que regrabar) y necesita **refresco** periódico (la carga se fuga con el tiempo).
- **Mealy**: salida = $f(\text{estado}, \text{entrada})$ — se escribe sobre las flechas. **Moore**: salida = $f(\text{estado})$ solamente — se escribe dentro del círculo.
- Metodología FSM: diagrama/tabla de estados (con TODAS las transiciones) $\rightarrow$ codificación binaria $\rightarrow$ tabla de transiciones y salidas $\rightarrow$ ecuaciones de excitación de los FF $\rightarrow$ Karnaugh (con don't care si sobran códigos) $\rightarrow$ circuito.